

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №7.

Тема: «Нелинейные уравнения»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 18

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2025

I. Метод половинного деления

1. Постановка задачи

Метод половинного деления

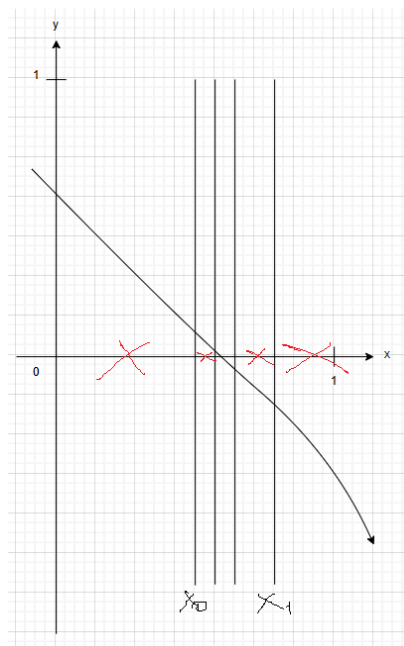
$$\arccos x - \sqrt{1 - 0,3x^3} = 0$$

Отрезок, содержащий корень: $[0;1]$

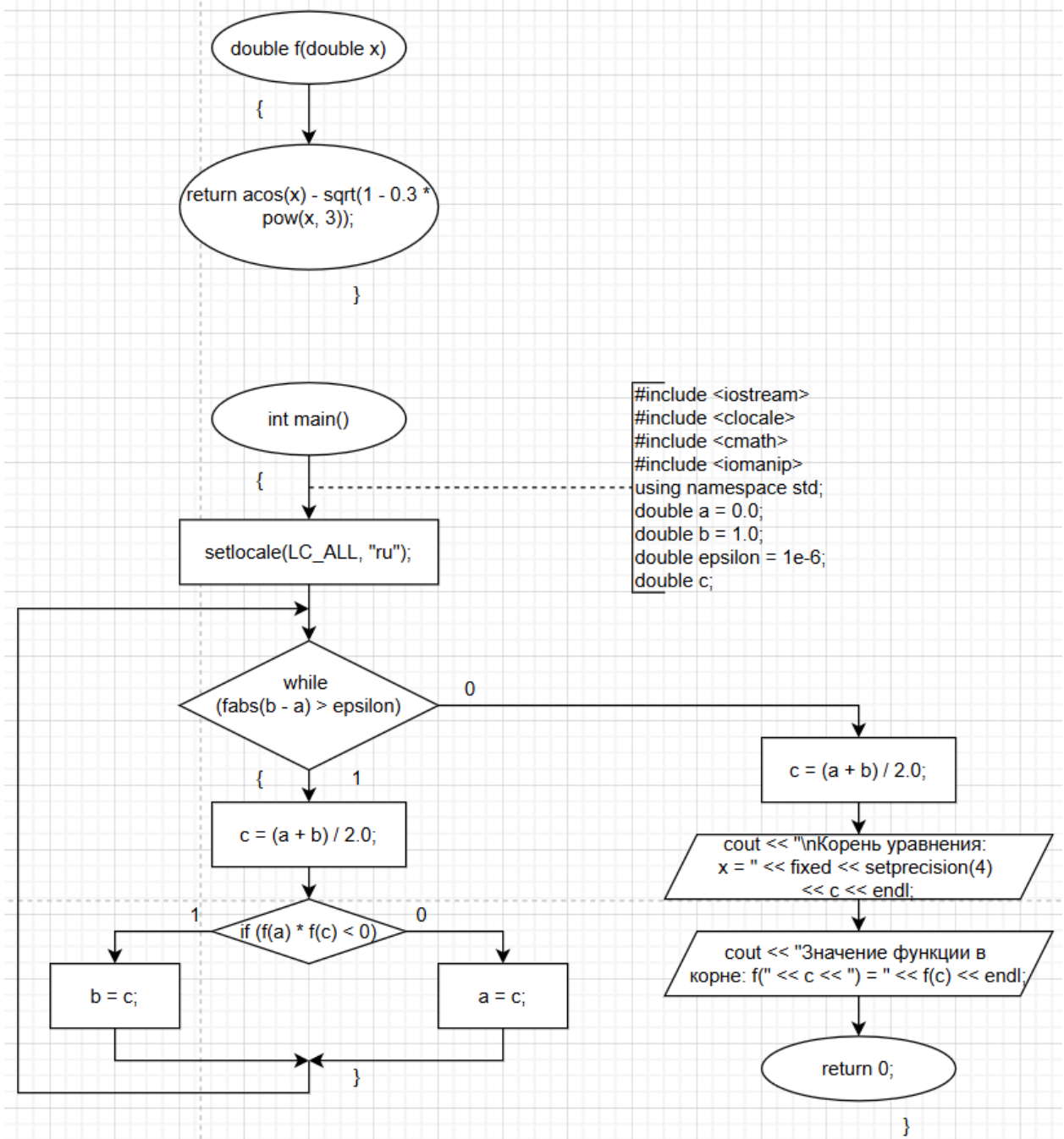
Точное значение: 0,5629

2. Анализ задачи

1. Проверяем непрерывность функции на интервале $[a;b]$ – $f(x)$ не имеет разрывов на интервале $[a;b]$
2. Проверяем монотонность функции на интервале $[a;b]$ – $f'(x) < 0$ на всём интервале $[a;b]$, поэтому функция монотонна
3. Проверяем, что уравнение на интервале $[a;b]$ имеет хотя бы один корень – $f(a) \cdot f(b) < 0$, в нашем случае $f(a) > 0$, $f(b) < 0$, следовательно, $f(a) \cdot f(b) < 0$, наша функция имеет корень
4. Берём точку x_i на середине интервала $[a;b]$ – $x_i = (a+b)/2$
5. Проверяем знак произведения $f(a) \cdot f(x_i)$; если оно отрицательно, корень лежит в левой половине, иначе — в правой; в нашем случае выбор подынтервала определяется условием $\text{if } (f(a) * f(c) < 0)$
6. Выбираем интервал с отрицательным произведением и переносим границу a или b в точку x_i , в нашем случае обновление границ происходит внутри цикла в зависимости от знака произведения
7. Повторяем процесс до тех пор, пока $|b-a| > \varepsilon$ (ε – точность вычисления, $\varepsilon = 0,000001$), после чего берём $x = (a+b)/2$ за приближённое значение корня



3. Блок-схема



4. Код

```

#include <iostream>
#include <locale>
#include <cmath>
#include <iomanip>
#include <chrono>

using namespace std;
using namespace std::chrono;

double f(double x) {
    return acos(x) - sqrt(1.0 - 0.3 * x * x * x);
}

int main() {

```

```

setlocale(LC_ALL, "ru");

double a = 0.0, b = 1.0;
const double epsilon = 1e-6;
int iterations = 0;

cout << "Решение уравнения  $\arccos(x) - \sqrt{1 - 0.3x^3} = 0$ " << endl;
cout << "Метод половинного деления" << endl;
cout << "Отрезок: [" << a << "; " << b << "]" << endl;
cout << "Точность: " << scientific << epsilon << endl << endl;

auto start = high_resolution_clock::now();

if (f(a) * f(b) >= 0) {
    cout << "На отрезке [" << a << "; " << b << "] нет корня или их  
несколько." << endl;
}
else {
    while (fabs(b - a) > epsilon) {
        double c = (a + b) / 2.0;
        iterations++;
        if (f(a) * f(c) < 0) {
            b = c;
        }
        else {
            a = c;
        }
    }

    double root = (a + b) / 2.0;
    auto end = high_resolution_clock::now();
    double time_spent = duration<double, milli>(end - start).count();

    cout << "\nКорень уравнения: x = " << fixed << setprecision(4) << root  
<< endl;
    cout << "Значение функции в корне: f(" << fixed << setprecision(6) <<  
root << ") = "  
        << scientific << setprecision(4) << f(root) << endl;
    cout << "Количество итераций: " << iterations << endl;
    cout << "Время выполнения: " << fixed << setprecision(3) << time_spent  
<< " мс" << endl;
}

return 0;
}

```

5. Результат выполнения программы

```

Решение уравнения  $\arccos(x) - \sqrt{1 - 0.3x^3} = 0$ 
Метод половинного деления
Отрезок: [0; 1]
Точность: 1.000000e-06

Корень уравнения: x = 0.5629
Значение функции в корне: f(0.562926) = 1.4597e-07
Количество итераций: 20
Время выполнения: 0.016 мс

```

II. Метод Ньютона

1. Постановка задачи

Метод Ньютона

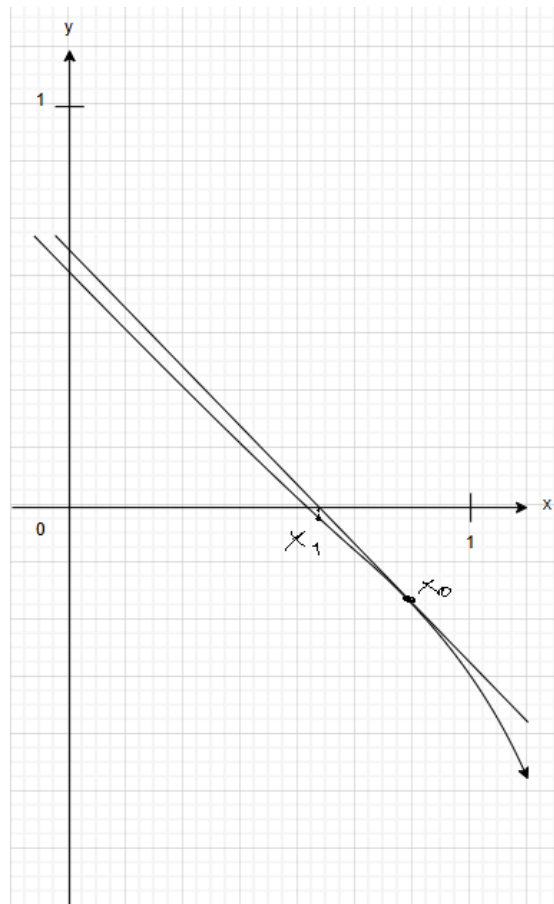
$$\arccos x - \sqrt{1 - 0,3x^3} = 0$$

Отрезок, содержащий корень: $[0;1]$

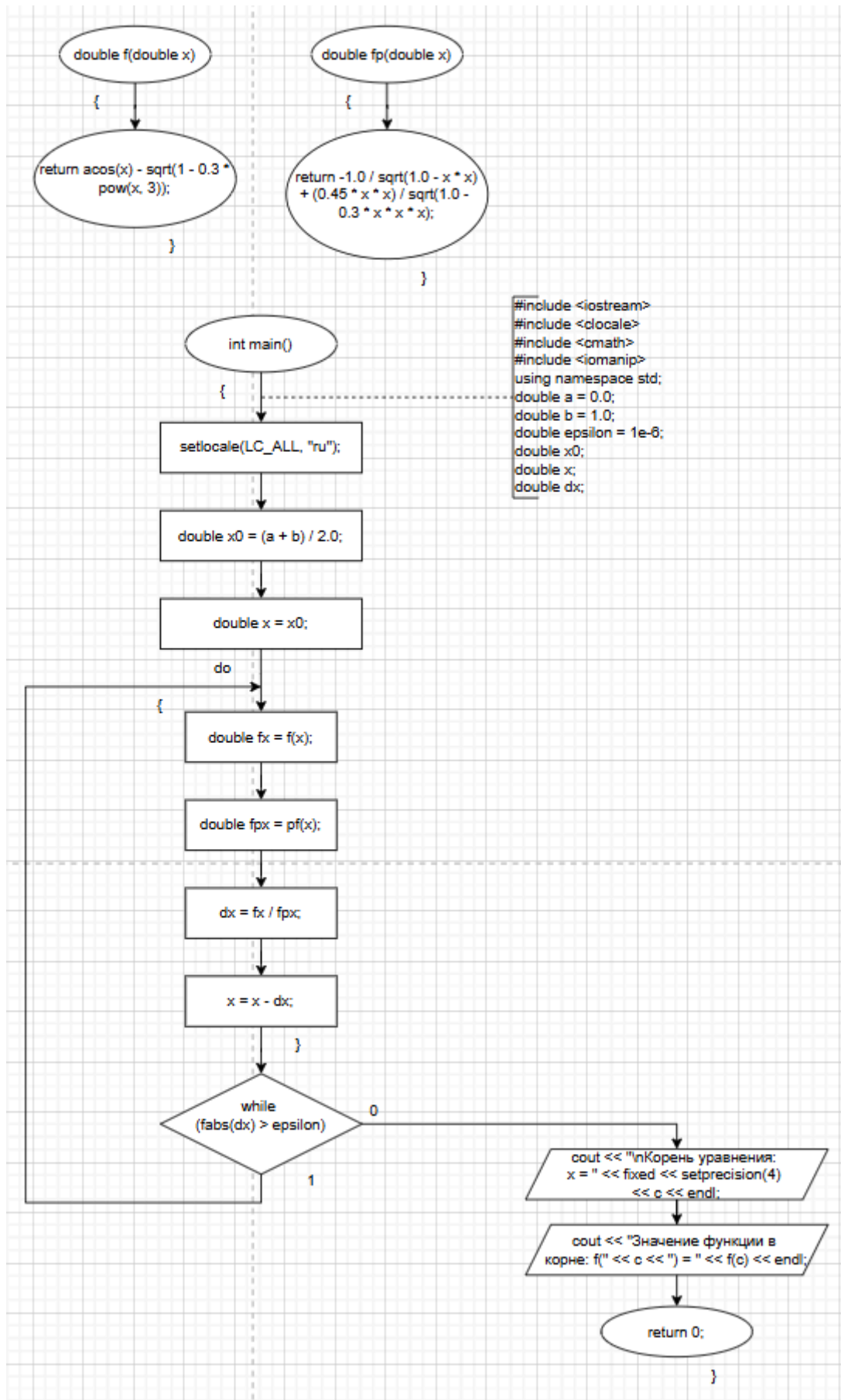
Точное значение: 0,5629

2. Анализ задачи

1. Проверяем непрерывность функции на интервале $[a;b]$ – $f(x)$ не имеет разрывов на интервале $[a;b]$
2. Проверяем монотонность функции на интервале $[a;b]$ – $f'(x) < 0$ на всём интервале $[a;b]$, поэтому функция монотонна
3. Проверяем, вогнута или выпукла функция – $f''(x) < 0$ на $[a;b]$, следовательно, функция выпукла вверх
4. Выбираем начальное приближение x_1 на интервале $[a;b]$ с учётом знака функции и её выпуклости; в нашем случае берём середину отрезка: $x_1 = (a+b)/2$
5. Находим следующую точку x_i по формуле: $x_i = x_{i-1} - f(x_{i-1})/f'(x_{i-1})$
6. Повторяем процесс до тех пор, пока $|x_i - x_{i-1}| > \varepsilon$ (ε – точность вычисления, $\varepsilon = 0,000001$), после чего берём x_i за приближённое значение корня



3. Блок-схема



4. Код

```
#include <iostream>
#include <locale>
#include <cmath>
#include <iomanip>
#include <chrono>

using namespace std;
using namespace std::chrono;

double f(double x) {
    return acos(x) - sqrt(1.0 - 0.3 * x * x * x);
}

// Производная функции f(x)
double pf(double x) {
    return -1.0 / sqrt(1.0 - x * x) + (0.45 * x * x) / sqrt(1.0 - 0.3 * x * x * x);
}

int main() {
    setlocale(LC_ALL, "ru");

    double a = 0.0, b = 1.0;
    const double epsilon = 1e-6;
    int iterations = 0;

    cout << "Решение уравнения  $\arccos(x) - \sqrt{1 - 0.3x^3} = 0$ " << endl;
    cout << "Метод Ньютона (касательных)" << endl;
    cout << "Отрезок: [" << a << "; " << b << "]" << endl;
    cout << "Точность: " << scientific << epsilon << endl << endl;

    auto start = high_resolution_clock::now();

    // Выбираем начальное приближение
    // В данном случае можно взять середину отрезка или точку, где функция
    // меняет знак
    double x0 = (a + b) / 2.0; // Начальное приближение

    if (f(a) * f(b) >= 0) {
        cout << "На отрезке [" << a << "; " << b << "] нет корня или их
несколько." << endl;
    }
    else {
        double x = x0;
        double dx;
        do {
            double fx = f(x);
            double fpx = pf(x);

            // Проверка на деление на ноль
            if (fabs(fpx) < 1e-12) {
                cout << "Производная близка к нулю. Метод Ньютона не применим в
этой точке." << endl;
                break;
            }

            dx = fx / fpx;
            x = x - dx;
            iterations++;
        } while (fabs(dx) > epsilon);

        auto end = high_resolution_clock::now();
        double time_spent = duration<double, milli>(end - start).count();
    }
}
```

```

        cout << "\nКорень уравнения: x = " << fixed << setprecision(4) << x <<
endl;
        cout << "Значение функции в корне: f(" << fixed << setprecision(6) << x
<< ") = "
            << scientific << setprecision(4) << f(x) << endl;
        cout << "Количество итераций: " << iterations << endl;
        cout << "Время выполнения: " << fixed << setprecision(3) << time_spent
<< " мс" << endl;
    }

    return 0;
}

```

5. Результат выполнения программы

```

Решение уравнения  $\arccos(x) - \sqrt{1 - 0.3 \cdot x^3} = 0$ 
Метод Ньютона (касательных)
Отрезок: [0; 1]
Точность: 1.000000e-06

```

```

Корень уравнения: x = 0.5629
Значение функции в корне: f(0.562926) = -1.9984e-15
Количество итераций: 3
Время выполнения: 0.031 мс

```


III. Метод итераций

1. Постановка задачи

Метод итераций

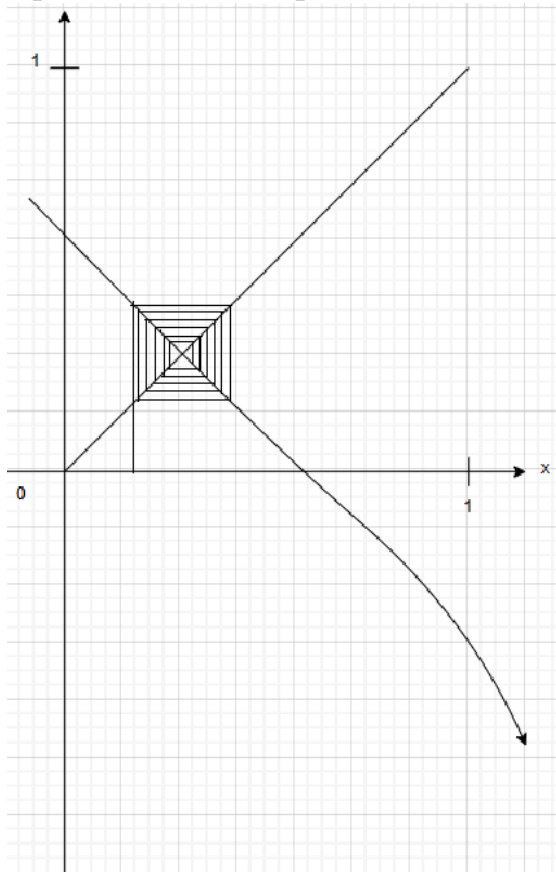
$$\arccos x - \sqrt{1 - 0,3x^3} = 0$$

Отрезок, содержащий корень: $[0;1]$

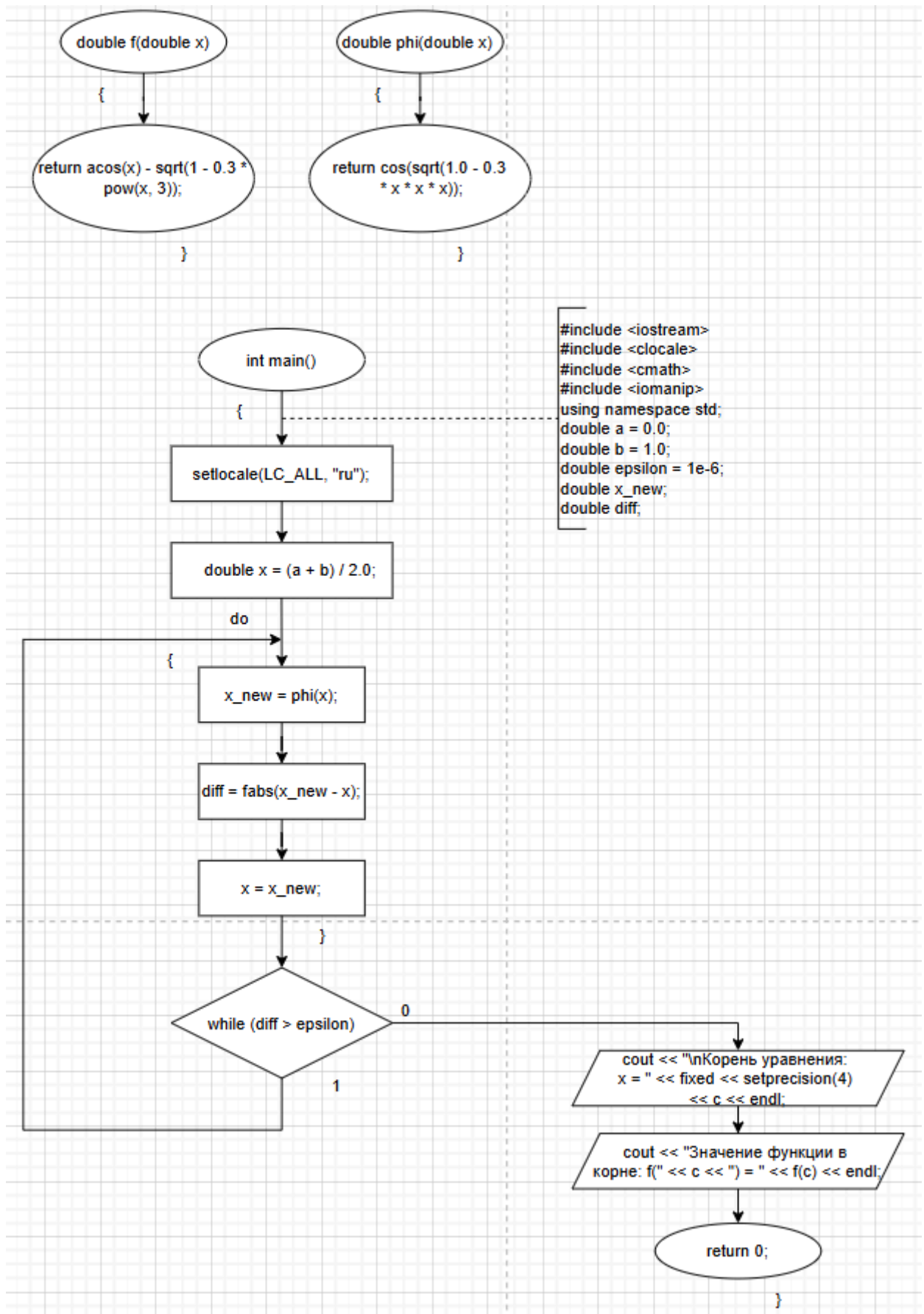
Точное значение: 0,5629

2. Анализ задачи

1. Проверяем, что уравнение на интервале $[a;b]$ имеет хотя бы один корень — $f(a) \cdot f(b) < 0$
2. Проверяем непрерывность функции на интервале $[a;b]$ — $f(x)$ не имеет разрывов на интервале $[a;b]$
3. Проверяем монотонность функции на интервале $[a;b]$ — $f'(x) < 0$ на всём интервале, следовательно, функция строго убывает и имеет единственный корень
4. Преобразуем исходное уравнение $\arccos x = \sqrt{1 - 0,3x^3}$ к виду $x = \varphi(x)$, где $\varphi(x) = \cos(\sqrt{1 - 0,3x^3})$
5. Выбираем начальное приближение x_0 внутри отрезка $[a;b]$ — в нашем случае $x_0 = (a+b)/2$
6. Вычисляем очередное приближение по формуле: $x_i = \varphi(x_{i-1})$
7. Повторяем процесс до тех пор, пока $|x_i - x_{i-1}| > \varepsilon$ ($\varepsilon = 0,000001$ — заданная точность), после чего принимаем x_i за приближённое значение корня



3. Блок-схема



4. Код

```
#include <iostream>
#include <locale>
#include <cmath>
#include <iomanip>
#include <chrono>

using namespace std;
using namespace std::chrono;

double f(double x) {
    return acos(x) - sqrt(1.0 - 0.3 * x * x * x);
}

// Итерационная функция:  $x = \cos(\sqrt{1 - 0.3x^3})$ 
double phi(double x) {
    return cos(sqrt(1.0 - 0.3 * x * x * x));
}

int main() {
    setlocale(LC_ALL, "ru");

    double a = 0.0, b = 1.0;
    const double epsilon = 1e-6;
    const int max_iter = 100000;
    int iterations = 0;

    cout << "Решение уравнения  $\arccos(x) - \sqrt{1 - 0.3x^3} = 0$ " << endl;
    cout << "Метод простых итераций" << endl;
    cout << "Отрезок: [" << a << "; " << b << "]" << endl;
    cout << "Точность: " << scientific << epsilon << endl << endl;

    auto start = high_resolution_clock::now();

    if (f(a) * f(b) >= 0) {
        cout << "На отрезке [" << a << "; " << b << "] нет корня или их  
несколько." << endl;
        return 0;
    }

    double x = (a + b) / 2.0; // начальное приближение
    double x_new;
    double diff;

    do {
        x_new = phi(x);
        diff = fabs(x_new - x);
        x = x_new;
        iterations++;

        if (iterations > max_iter) {
            cout << "Достигнуто максимальное число итераций. Процесс не  
сошёлся." << endl;
            return 1;
        }
    } while (diff > epsilon);

    auto end = high_resolution_clock::now();
    double time_spent = duration<double, milli>(end - start).count();

    cout << "\nКорень уравнения:  $x =$  " << fixed << setprecision(4) << x << endl;
    cout << "Значение функции в корне:  $f($ " << fixed << setprecision(6) << x <<
    ") = "
```

```

        << scientific << setprecision(4) << f(x) << endl;
    cout << "Количество итераций: " << iterations << endl;
    cout << "Время выполнения: " << fixed << setprecision(3) << time_spent << "
мс" << endl;

    return 0;
}

```

5. Результат выполнения программы

```

Решение уравнения  $\arccos(x) - \sqrt{1 - 0.3x^3} = 0$ 
Метод простых итераций
Отрезок: [0; 1]
Точность: 1.000000e-06

```

```

Корень уравнения:  $x = 0.5629$ 
Значение функции в корне:  $f(0.562926) = 2.2517e-08$ 
Количество итераций: 7
Время выполнения: 0.014 мс

```

Выводы

1. Точность результата

- Метод Ньютона показал наибольшую точность (10^{-15}).
 - Метод простых итераций — хорошая точность (10^{-8}), но уступает Ньютону.
 - Метод половинного деления — удовлетворительная точность (10^{-7}).
-

2. Количество итераций

- Метод Ньютона — всего 3 итерации.
 - Метод простых итераций — 7 итераций.
 - Метод половинного деления — 20 итераций — самый медленный по количеству шагов.
-

3. Время выполнения

- Метод простых итераций — самый быстрый по времени (0.014 мс).
- Метод половинного деления — 0.016 мс — тоже очень быстр.
- Метод Ньютона — самый медленный по времени (0.031 мс), несмотря на минимальное число итераций.